JAVA COURSE

{ variabili ed operatori }



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.1

INDICE

01 Variabili

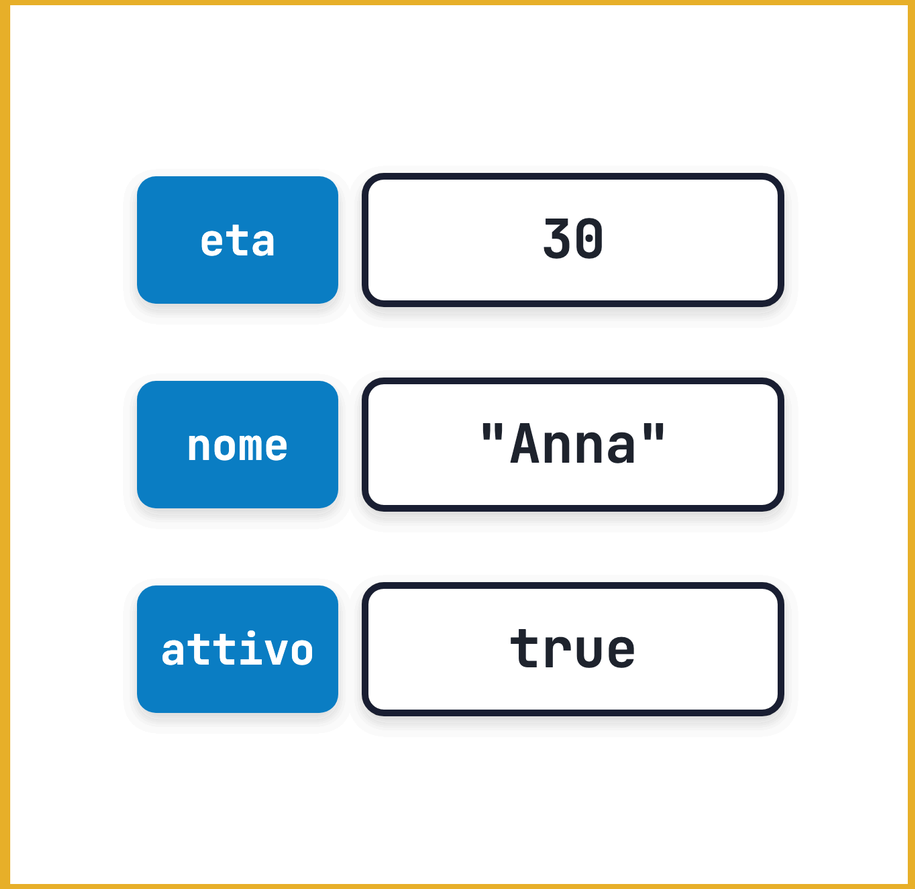
02 Tipi di dato

03 Conversioni di tipo

04 Operatori

05 Numeri in virgola mobile

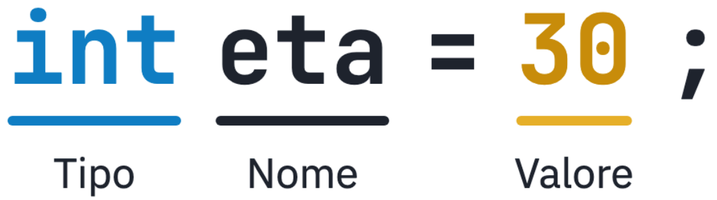
01
VARIABILI



COS'È UNA VARIABILE

Una **variabile** è uno spazio di memoria con un nome, che contiene un valore. Il valore può cambiare durante l'esecuzione

- ***Tipo:*** *stabilisce quali valori può contenere*
- ***Nome:*** *identifica la variabile nel codice*
- ***Valore:*** *il dato contenuto in quel momento*



DICHIARAZIONE E INIZIALIZZAZIONE

Prima di usare una variabile bisogna **dichiararla**, con tipo e nome.

Inizializzarla significa assegnarle il primo valore

```
int eta;                    // dichiarazione
eta = 30;                   // inizializzazione

int anni = 30;              // dichiarazione e inizializzazione
int ore = 11, minuti = 59;   // più variabili dello stesso tipo
```

VARIABILE NON INIZIALIZZATA

Una variabile va dichiarata e inizializzata prima di essere letta, altrimenti javac si ferma

Terminale

```
$ javac Eta.java
Eta.java:4: error: variable eta might not have been initialized
        System.out.println(eta);
                           ^
1 error
```

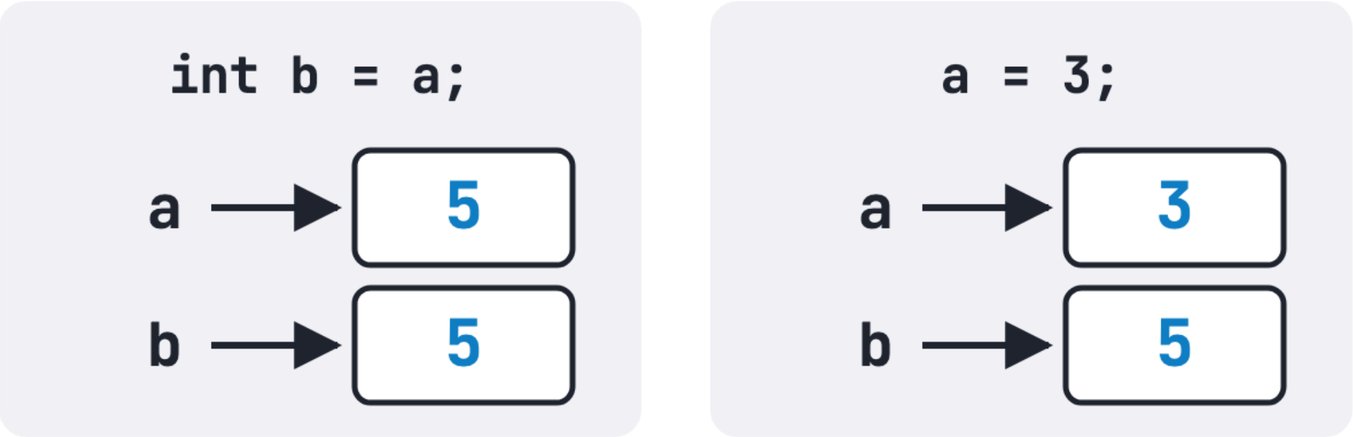
L'ASSEGNAZIONE

Il simbolo `=` non indica un'uguaglianza: copia il valore di destra nella variabile di sinistra

```
int a = 5;
int b = a;     // b vale 5: il valore di a viene copiato
a = 3;         // a vale 3, b vale ancora 5
a = a + 1;     // a vale 4
```

LO STATO DEL PROGRAMMA

Le variabili e i loro valori in un certo istante formano lo **stato** del programma, rappresentabile come **diagramma di stato**



Lo stato dopo ciascuna istruzione

I NOMI DELLE VARIABILI

Il nome di una variabile, detto **identificatore**, segue regole precise: se non le rispetta, il codice non compila

- ***Primo carattere:*** *una lettera, _ o $, mai una cifra*
- ***Vietati:*** *spazi, trattini, parole riservate*
- ***Case sensitive:*** *eta, Eta ed ETA sono tre nomi diversi*

```
eta             // ok
nomeCompleto    // ok
anno2026        // ok
2anni           // errore
data-nascita    // errore
class           // errore
```

LE CONVENZIONI DEI NOMI

Le convenzioni non sono regole del compilatore, ma linee guida per gli sviluppatori

- ***Nomi significativi:*** *eta e prezzoTotale, non x e pt*

Si usa per	Stile	Esempio
variabili e metodi	**camelCase**	**nomeCompleto**
classi	PascalCase	ContoCorrente
costanti	SCREAMING_SNAKE_CASE	MAX_STUDENTI
package	minuscolo con punti	com.example.banca

LE PAROLE RISERVATE

Le parole chiave fanno parte della grammatica del linguaggio e non si possono usare come nomi

Categoria	Esempi	Come nome
Parole riservate	class, public, static, void, int, if, new	vietate
Parole contestuali	var, record, yield, sealed	ammesse, ma sconsigliate
Letterali	true, false, null	vietati

LE COSTANTI

Una variabile `final` riceve un valore una sola volta: una nuova assegnazione è un errore di compilazione

```
final double IVA = 0.22;
final int MAX_STUDENTI = 30;

IVA = 0.10;   // errore: cannot assign a value to final variable IVA
```

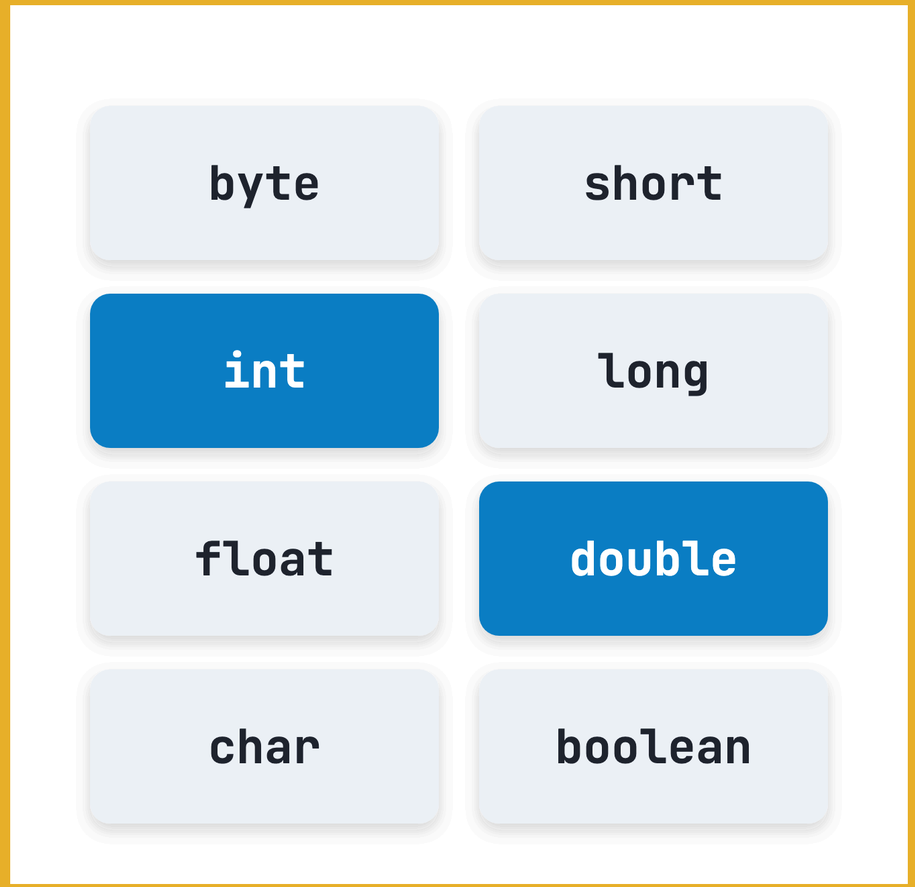
VAR

Da Java 10, con `var` il compilatore deduce il tipo di una variabile locale dal valore iniziale. Il tipo poi non cambia più

```
var eta = 30;           // int
var prezzo = 9.99;      // double
var nome = "Anna";      // String

eta = "trenta";         // errore: String cannot be converted to int
var x;                  // errore: cannot infer type for local variable x
```

02
TIPI DI DATO



IL TIPO DI UN DATO

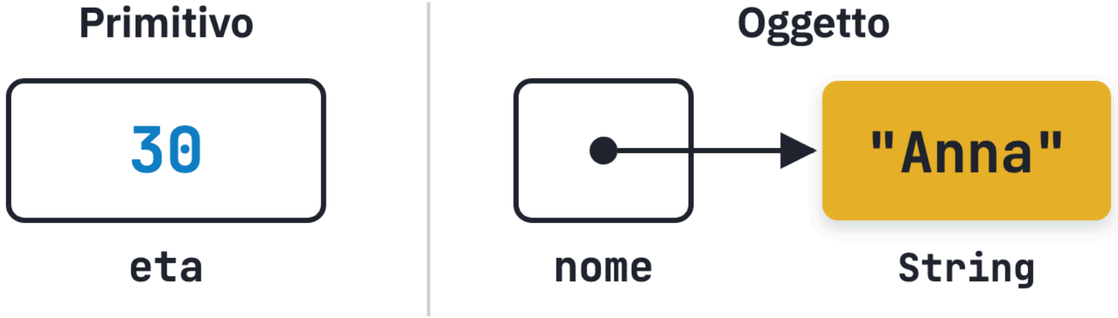
Il **tipo** di una variabile stabilisce quali valori può contenere, quanta memoria occupa e quali operazioni ammette

Java è tipizzato staticamente: javac controlla i tipi prima dell'esecuzione. Un int contiene solo numeri interi, occupa 32 bit e si può sommare a un altro int; un boolean no

PRIMITIVI E OGGETTI

I tipi di Java si dividono in due famiglie: **primitivi** e **reference**

- ***Primitivo:*** *la variabile contiene direttamente il valore*
- ***Reference:*** *la variabile contiene un riferimento all'oggetto in memoria, come una String*
- ***null:*** *il valore di un reference che non indica nessun oggetto*



I TIPI PRIMITIVI

Otto tipi primitivi, in quattro categorie: interi, decimali, caratteri e booleani

Tipo	Bit	Valori
byte	8	da -128 a 127
short	16	da -32.768 a 32.767
int	**32**	**circa ± 2,1 miliardi**
long	64	circa ± $9{,}2 \times 10^{18}$
float	32	decimali, circa 7 cifre significative
double	**64**	**decimali, circa 15-16 cifre significative**
char	16	una code unit UTF-16 (un carattere BMP)
boolean	–	true o false

GLI INTERI

Quattro tipi per i numeri interi: cambia la memoria occupata, e quindi l'intervallo dei valori

- ***int:*** *la scelta predefinita*
- ***long:*** *per valori oltre i due miliardi*
- ***byte e short:*** *per risparmiare memoria con grandi quantità di dati*

```
int studenti = 25;
long abitantiTerra = 8_100_000_000L;

System.out.println(Integer.MAX_VALUE);    // 2147483647
System.out.println(Long.MAX_VALUE);       // 9223372036854775807
```

I LETTERALI NUMERICI

Un **letterale** è un valore scritto direttamente nel codice. Il suo tipo dipende da come è scritto

- ***Intero:*** *è un int*
- ***Con la virgola:*** *è un double*
- ***Suffissi:*** *L per long, f per float*
- ***Underscore:*** *separa le cifre, solo per leggibilità*

```
42              // int
42L             // long
3.14            // double
3.14f           // float
1_000_000       // int, un milione
0xFF            // int esadecimale: 255
0b1010          // int binario: 10
3000000000      // errore: integer number too large
```

I DECIMALI

`float` e `double` rappresentano i numeri con la virgola. Nel codice il separatore decimale è il punto

- ***double:*** *la scelta predefinita, più preciso*
- ***float:*** *metà della memoria, letterale con f*

```
double prezzo = 19.99;
float temperatura = 21.5f;
float errore = 21.5;              // errore: possible lossy conversion from double to float

System.out.println(1.0 / 3);    // 0.3333333333333333
System.out.println(1.0f / 3);   // 0.33333334
```

I CARATTERI

Il tipo `char` contiene una singola **code unit UTF-16** a 16 bit. I caratteri del piano base (lettere, cifre, simboli comuni) occupano un `char`; un'emoji ne richiede due

- ***Apici:*** *'A' è un char, "A" è una String*
- ***Sequenze di escape:*** *'\n' a capo, '\t' tabulazione, '\'' apice, '\\' barra*

```
char iniziale = 'D';
char aCapo = '\n';

char a = 'A';
System.out.println(a + 1);   // 66: nel calcolo il char diventa un int
```

I BOOLEANI

Il tipo `boolean` ha solo due valori, true e false. È il risultato dei confronti

- ***Niente numeri:*** *0 e 1 non sono boolean*
- ***Dimensione:*** *la specifica di Java non la fissa*

```
boolean maggiorenne = true;
boolean promosso = false;

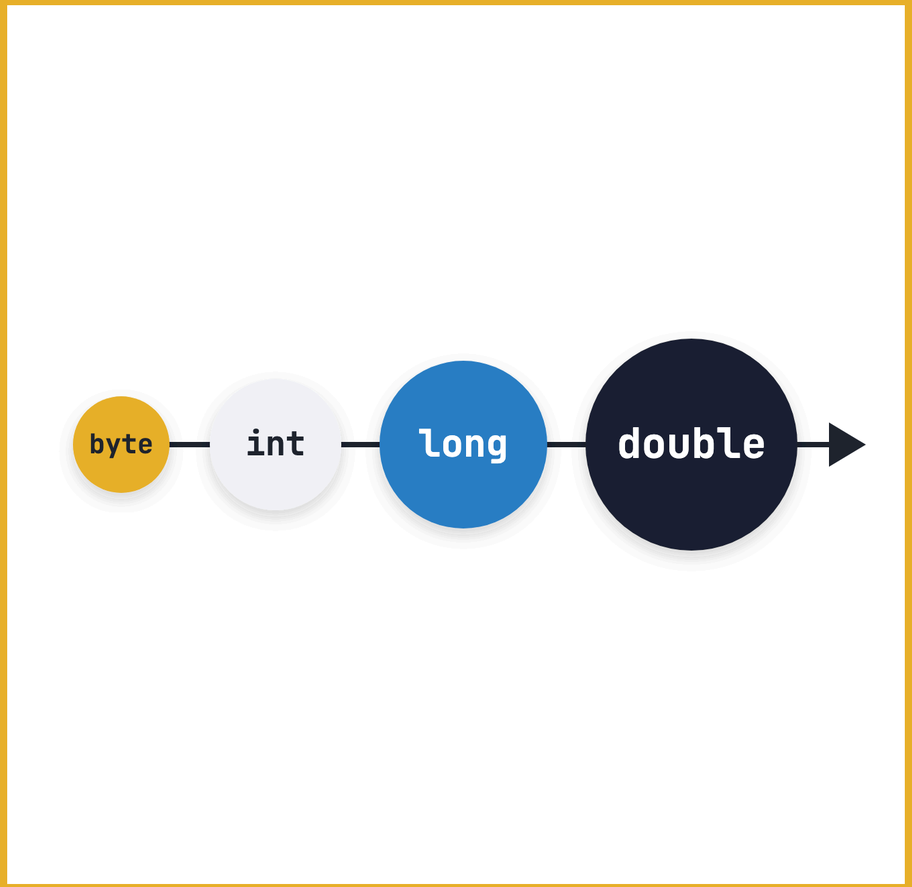
boolean b = 1;   // errore: int cannot be converted to boolean
```

I VALORI PREDEFINITI

I **campi** di una classe ricevono un valore predefinito. Le variabili locali no: vanno sempre inizializzate

Tipo del campo	Valore predefinito
byte, short, int, long	0
float, double	0.0
char	'\u0000', il carattere di codice zero
boolean	false
reference, come String	**null**

03
CONVERSIONI DI TIPO



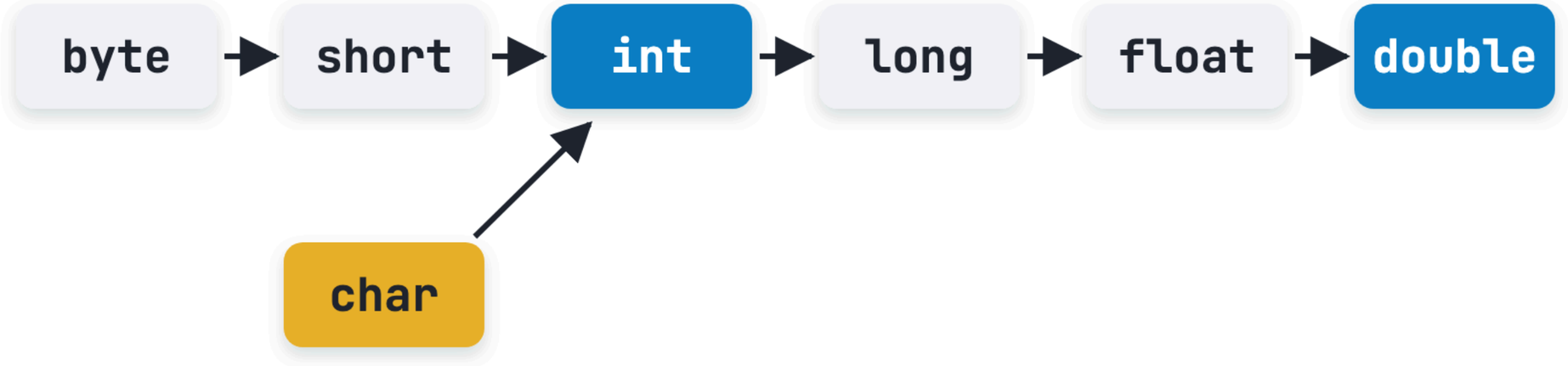
DUE TIPI DI CONVERSIONE

Spesso un valore di un tipo serve in una variabile di un altro tipo. Java distingue due casi

- ***Allargamento (widening):*** *verso un tipo più ampio; è automatico*
- ***Restringimento (narrowing):*** *verso un tipo più piccolo; serve un cast esplicito e si possono perdere dati*

L'ALLARGAMENTO

Java converte da solo un valore verso un tipo più ampio, seguendo le frecce. Un char diventa un int, e da lì può proseguire



ALLARGAMENTO: ESEMPI

L'allargamento non richiede nessun cast

- ***Precisione:*** *un float conserva circa 7 cifre significative, un int ne ha fino a 10; per questo da int a float le ultime cifre possono cambiare. Lo stesso vale da long, fino a 19 cifre, a double, circa 16*

```
int i = 100;
long l = i;             // 100
double d = l;           // 100.0
double y = 1;           // 1.0

int grande = 123456789;
float f = grande;       // 1.23456792E8: le ultime cifre cambiano
```

IL CAST

Per convertire verso un tipo più piccolo serve un **cast**: il tipo di destinazione tra parentesi tonde

- ***Sintassi:*** *(tipo) valore*
- ***Significato:*** *il programmatore accetta la possibile perdita di dati*

```
double prezzo = 9.99;

int intero = prezzo;          // errore: possible lossy conversion from double to int
int troncato = (int) prezzo;  // 9
```

LA PERDITA DI DATI

Un cast non arrotonda e non controlla l'intervallo

- ***Da decimale a intero:*** *si scarta la parte decimale*
- ***Tra interi:*** *si tengono solo i bit che il tipo può contenere*

```
System.out.println((int) 9.99);     // 9
System.out.println((int) -9.99);    // -9
System.out.println((byte) 200);     // -56
System.out.println((byte) 300);     // 44
```

LA PROMOZIONE NUMERICA

Prima di un calcolo, Java porta gli operandi a un tipo comune. Il risultato ha quel tipo

- ***Almeno int:*** *byte, short e char diventano int*
- ***Il più ampio:*** *con un long, un float o un double, anche l'altro operando diventa di quel tipo*

```
byte a = 10;
byte b = 20;
byte c = a + b;         // errore: a + b è un int
int d = a + b;          // 30
double e = 5 / 2.0;     // 2.5: 5 diventa double
```

PROMOZIONE: COMPILA?

Quali righe compilano?

```
short s1 = 1;
byte b1 = 1, b2 = 2;
short s2 = +s1;          // ??
int somma = b1 + b2;     // ??
byte b = s1 + 2;         // ??
float f = 1.0f;
double d = 1.0;
int x = f - 1;           // ??
double y = f + d;        // ??
float f2 = f + d;        // ??
```

COSTANTI E INTERVALLI

Il letterale 10 è un int, ma si può assegnare a un byte: il compilatore conosce il valore e verifica che rientri nell'intervallo

- ***Con una variabile:*** *il compilatore non ne conosce il valore, quindi serve un cast*

```
byte b = 10;      // ok: 10 rientra tra -128 e 127
byte c = 200;     // errore: 200 non rientra in un byte

int n = 10;
byte d = n;       // errore: possible lossy conversion from int to byte
```

04
OPERATORI



ESPRESSIONI E ISTRUZIONI

Un'**espressione** produce un valore. Un'**istruzione** è un'azione completa e termina con il punto e virgola

- ***Espressioni:*** *a + 5, a * 2, "Ciao " + nome*
- ***Istruzioni:*** *una dichiarazione, un'assegnazione, una chiamata a println*

```
int a = 10;                     // dichiarazione
a = a + 5;                      // assegnazione; a + 5 è un'espressione
System.out.println(a * 2);      // chiamata; a * 2 è un'espressione
a + 5;                          // errore: not a statement
```

GLI OPERATORI

Un **operatore** si applica a uno o più operandi e produce un valore

Famiglia	Operatori
Aritmetici	+ - * / %
Incremento e decremento	++ --
Assegnazione	= += -= *= /= %=
Concatenazione	+ con almeno una String
Relazionali	== != < > <= >=
Logici	&& || ! & | ^

GLI OPERATORI ARITMETICI

I cinque operatori aritmetici si applicano a tutti i tipi numerici e a char

- ***Tra interi:*** */ restituisce un intero, % il resto della divisione*

```
int a = 17, b = 5;
System.out.println(a + b);   // 22
System.out.println(a - b);   // 12
System.out.println(a * b);   // 85
System.out.println(a / b);   // 3
System.out.println(a % b);   // 2
```

LA DIVISIONE INTERA

Se entrambi gli operandi sono interi, la divisione è intera: la parte decimale si scarta

```
System.out.println(7 / 2);      // 3
System.out.println(7 / 2.0);    // 3.5

double media = 7 / 2;           // 3.0: la divisione avviene prima, tra interi
double esatta = (double) 7 / 2;   // 3.5
```

IL RESTO

L'operatore `%` restituisce il resto della divisione. Il risultato ha il segno del primo operando

- ***Pari o dispari:*** *n % 2 vale 0 se n è pari*
- ***Ultima cifra:*** *n % 10*
- ***Ore e minuti:*** *135 / 60 e 135 % 60*

```
17 % 5      // 2
10 % 2      // 0
1234 % 10   // 4
135 / 60    // 2
135 % 60    // 15
-17 % 5     // -2
5.5 % 2     // 1.5
```

L'ORDINE DEI CALCOLI

Come in matematica, `* / %` vengono prima di `+ -`. A parità di precedenza si procede da sinistra a destra

- ***Parentesi:*** *cambiano l'ordine e rendono chiaro il codice*

```
System.out.println(2 + 3 * 4);      // 14
System.out.println((2 + 3) * 4);    // 20
System.out.println(10 - 4 - 3);     // 3
System.out.println(20 / 2 * 5);     // 50
```

INCREMENTO E DECREMENTO

`++` e `--` aumentano o diminuiscono una variabile di 1

- ***Prefisso ++i:*** *prima incrementa, poi restituisce il valore*
- ***Postfisso i++:*** *prima restituisce il valore, poi incrementa*
- ***Da soli:*** *i++ e ++i hanno lo stesso effetto*

```
int i = 1;
int post = i++;   // post vale 1, i vale 2

i = 1;
int pre = ++i;    // pre vale 2, i vale 2
```

INCREMENTO: COSA STAMPA?

Che cosa stampano le tre istruzioni?

```
int x = 5;
System.out.println(++x);    // ??
System.out.println(x--);    // ??
System.out.println(x);      // ??
```

L'ASSEGNAZIONE COMPOSTA

Gli operatori composti uniscono un'operazione e un'assegnazione

```
int n = 10;
n += 5;     // n = n + 5     → 15
n -= 3;     // n = n - 3     → 12
n *= 2;     // n = n * 2     → 24
n /= 5;     // n = n / 5     → 4
n %= 3;     // n = n % 3     → 1
```

IL CAST NASCOSTO

L'assegnazione composta contiene un cast implicito verso il tipo della variabile

- ***b += 1:*** *equivale a b = (byte) (b + 1)*
- ***Rischio:*** *il cast nascosto può far perdere dati senza errori*

```
byte b = 10;
b = b + 1;      // errore: possible lossy conversion from int to byte
b += 1;         // 11
b += 300;       // compila, ma b vale 55

int n = 5;
n *= 2.5;       // compila: n vale 12
```

L'OVERFLOW

Se un risultato supera l'intervallo del tipo, il valore riparte dall'estremo opposto. Java non segnala nessun errore

```
int max = Integer.MAX_VALUE;
System.out.println(max);        // 2147483647
System.out.println(max + 1);    // -2147483648

long l = max + 1;               // -2147483648: il calcolo avviene tra int
long ok = max + 1L;             // 2147483648
```

LA DIVISIONE PER ZERO

Il risultato dipende dal tipo degli operandi

- ***Tra interi:*** *il programma si interrompe con un'ArithmeticException*
- ***Con i decimali:*** *il risultato è Infinity, come si vedrà più avanti nel corso*

Terminale

```
$ java DivisionByZero
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at DivisionByZero.main(DivisionByZero.java:3)
```

LA CONCATENAZIONE

Se almeno un operando è una String, `+` unisce i due valori in una nuova stringa. Si procede da sinistra a destra: finché entrambi gli operandi sono numeri, + è una somma

- ***Altri operatori:*** *- * / non si applicano alle stringhe*

```
String nome = "Anna";
int eta = 30;
System.out.println(nome + " ha " + eta + " anni");   // Anna ha 30 anni
System.out.println(1 + 2 + "Hello");     // 3Hello
System.out.println("Hello" + 1 + 2);     // Hello12
System.out.println("Hello" + (1 + 2));   // Hello3
```

CONCATENAZIONE: COSA STAMPA?

Che cosa stampano le tre istruzioni?

```
String s = "hello";
int i = 1;
char c = 'A';

System.out.println(s + 1 + i);      // ??
System.out.println(c + 1 + "B");    // ??
System.out.println("B" + c + 1);    // ??
```

GLI OPERATORI RELAZIONALI

Operatore	Significato	Con x = 5	Risultato
==	uguale a	x == 5	true
!=	diverso da	x != 5	false
>	maggiore di	x > 3	true
<	minore di	x < 3	false
>=	maggiore o uguale a	x >= 5	true
<=	minore o uguale a	x <= 4	false

CONFRONTARE LE STRINGHE

Con le stringhe `==` non confronta il testo, ma i riferimenti. Il testo si confronta con `equals`

- ***equals:*** *true se il testo è lo stesso*
- ***==:*** *il motivo si vedrà con gli oggetti; per ora, con le stringhe non si usa*

```
String a = "Java";
String parte = "Ja";
String b = parte + "va";

System.out.println(a == b);         // false
System.out.println(a.equals(b));    // true
```

GLI OPERATORI LOGICI

Combinano valori boolean e restituiscono un boolean

- ***&& (and):*** *true se entrambi sono true*
- ***|| (or):*** *true se almeno uno è true*
- ***! (not):*** *inverte il valore*

a	b	a && b	a || b	!a
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

IL CORTO CIRCUITO

`&&` e `||` valutano il secondo operando solo se serve

- ***false && ...:*** *è sempre false, il secondo operando non si valuta*
- ***true || ...:*** *è sempre true*
- ***Uso tipico:*** *proteggere un'operazione che potrebbe fallire*

```
int x = 0;
boolean ok = x != 0 && 10 / x > 1;   // false, nessuna divisione
```

&, | E ^

Con operandi boolean, `&` e `|` sono and e or senza corto circuito. `^` è l'or esclusivo

- ***& e |:*** *valutano sempre entrambi gli operandi*
- ***^ (xor):*** *true se i due valori sono diversi*

```
boolean a = true, b = false;
System.out.println(a & b);   // false
System.out.println(a ^ b);   // true
System.out.println(a ^ a);   // false

int x = 0;
boolean ko = x != 0 & 10 / x > 1;   // ArithmeticException: / by zero
```

COMBINARE LE CONDIZIONI

Tra gli operatori logici, `!` viene prima di `&&`, che viene prima di `||`

- ***Intervalli:*** *18 <= eta <= 65 non compila; si scrive con due confronti*

```
int eta = 20;
boolean studente = true;

boolean sconto = eta < 26 && studente;          // true
boolean adulto = eta >= 18 && eta <= 65;        // true
boolean r1 = true || false && false;             // true: && prima di ||
boolean r2 = (true || false) && false;           // false
```

LA PRECEDENZA

Dall'alto verso il basso: un operatore più in alto si valuta prima

Gruppo	Operatori
Postfissi	x++ x--
Unari e cast	++x --x +x -x ! (tipo)
Moltiplicativi	* / %
Additivi	+ -
Relazionali	< > <= >=
Uguaglianza	== !=
Logici senza corto circuito	&, poi ^, poi |
Logici con corto circuito	&&, poi ||
Assegnazione	= += -= *= /= %=

L'ASSOCIATIVITÀ

A parità di precedenza decide l'**associatività**: quasi tutti gli operatori si raggruppano da sinistra a destra

- ***Da sinistra:*** *10 - 4 - 3 vale (10 - 4) - 3*
- ***Da destra:*** *gli operatori unari e le assegnazioni*

```
System.out.println(10 - 4 - 3);     // 3
System.out.println(10 - (4 - 3));   // 9

int a, b, c;
a = b = c = 5;     // a = (b = (c = 5)): tutte valgono 5
```

PRECEDENZA: COSA SUCCEDE?

Che valore hanno le variabili? Quale riga non compila?

```
int i = 0;
byte b = (byte) i + 1;          // ??
int x = 2 + 3 * 4 % 5;          // ??
boolean r = 5 > 3 == true;      // ??
String s = 1 + 2 + "3" + 4 + 5;   // ??
```

LA CLASSE MATH

Per i calcoli che non hanno un operatore, la libreria standard offre la classe `Math`, sempre disponibile senza import

Chiamata	Risultato	Significato
Math.abs(-5)	5	valore assoluto
Math.pow(2, 10)	1024.0	potenza
Math.sqrt(16)	4.0	radice quadrata
Math.max(3, 7)	7	massimo, e min per il minimo
Math.round(2.5)	3	arrotondamento
Math.floor(2.7)	2.0	per difetto, e ceil per eccesso
Math.PI	3.14159...	pi greco

MATH: ESEMPI

pow e sqrt restituiscono sempre un double

- ***Math.random():*** *un double casuale tra 0 incluso e 1 escluso*
- ***Math.addExact:*** *somma due interi e segnala l'overflow con un'eccezione*

```
double area = Math.PI * Math.pow(5, 2);          // 78.53981633974483
double ipotenusa = Math.sqrt(3 * 3 + 4 * 4);     // 5.0
double prezzo = Math.round(19.987 * 100) / 100.0;   // 19.99
int dado = (int) (Math.random() * 6) + 1;        // da 1 a 6
int somma = Math.addExact(Integer.MAX_VALUE, 1); // ArithmeticException: integer overflow
```

05

NUMERI IN VIRGOLA MOBILE

```
0.1 + 0.2
```

```
0.30000000000000004
```



UN CALCOLO SBAGLIATO

Dieci volte 0.1 dovrebbe fare 1. Con i double non sempre: dipende da come il computer rappresenta i numeri decimali

```
System.out.println(0.1 * 10);   // 1.0
System.out.println(0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1);   // 0.9999999999999999
System.out.println(0.1 + 0.2);  // 0.30000000000000004
```

FRAZIONE ED ESPONENTE

Un numero in virgola mobile si scrive come segno, frazione ed esponente, come nella notazione scientifica

- ***Nel computer:*** *la base è 2, non 10*
- ***Vantaggio:*** *numeri molto grandi e molto piccoli con pochi bit*

$$123.45 = +1.2345 \times 10^{2}$$

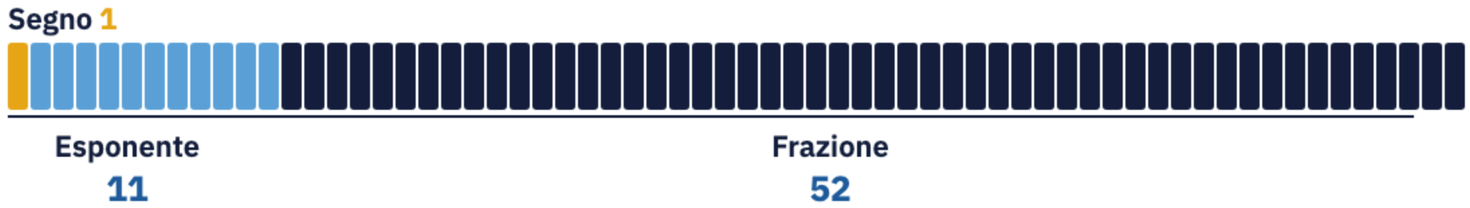
Segno | Frazione | Esponente

LO STANDARD IEEE 754

Lo standard **IEEE 754** stabilisce come i bit si dividono tra segno, esponente e frazione (significand)

- ***Bit implicito:*** *un* `1` *iniziale non memorizzato; la precisione effettiva è 24 bit per* `float` *e 53 per* `double`

Tipo	Bit	Esponente	Frazione	Cifre significative
float	32	8	23 (+1)	circa 7
double	**64**	**11**	**52 (+1)**	**circa 15-16**



PERCHÉ 0.1 NON È ESATTO

In base 10, 1/3 non ha una rappresentazione finita. In base 2 succede lo stesso a 0.1: il computer tiene i bit che può e scarta il resto

La differenza tra il numero voluto e quello memorizzato è l'**errore di arrotondamento**. Sono esatti solo gli interi non troppo grandi e le frazioni come 0.5 e 0.25

```
1/3 in base 10    =    0.3333333333…
0.1 in base 2     =    0.0001100110011…
```

CONFRONTARE I DECIMALI

Per via dell'errore di arrotondamento, `==` tra decimali è inaffidabile. Si confronta la differenza con una tolleranza

- ***Math.abs(a - 0.3) < 1e-9:*** *se la distanza tra a e 0.3 è minore di un miliardesimo, i due numeri si considerano uguali*

```
double a = 0.1 + 0.2;

System.out.println(a == 0.3);                    // false
System.out.println(Math.abs(a - 0.3) < 1e-9);    // true
```

INFINITY E NAN

Con i decimali, alcune operazioni non interrompono il programma ma producono valori speciali

- ***NaN:*** *Not a Number; non è uguale a niente, nemmeno a se stesso*

```
System.out.println(10.0 / 0);         // Infinity
System.out.println(-10.0 / 0);        // -Infinity
System.out.println(0.0 / 0);          // NaN
System.out.println(Math.sqrt(-1));    // NaN

double n = 0.0 / 0;
System.out.println(n == n);           // false
```

QUALE TIPO USARE

double va bene per misure e calcoli scientifici. Per il denaro serve un tipo che rappresenti i decimali in modo esatto

- ***double:*** *la scelta predefinita per i decimali*
- ***float:*** *quando la memoria conta, come in grafica e nei giochi*
- ***BigDecimal:*** *importi, tassi e contabilità; si vedrà più avanti*

CONFRONTO CON PYTHON

Differenze su variabili e operatori rispetto a Python

Aspetto	Python	Java
Dichiarazione	`x = 5`, tipo libero	`int x = 5;`, tipo fisso
Interi	precisione arbitraria	limiti fissi, overflow silenzioso
Divisione	`7 / 2` = 3.5, `//` intera	`7 / 2` = 3, già intera
Resto negativo	`-17 % 5` = 3	`-17 % 5` = -2
Stringhe	`==` confronta il testo	`==` i riferimenti, `equals` il testo
Operatori logici	`and or not`	`&& || !`

ERRORI COMUNI

I messaggi reali di `javac`

Errore	Messaggio di javac
Variabile non inizializzata	`variable might not have been initialized`
Tipo sbagliato nell'assegnazione	`incompatible types`
Restringimento senza cast	`possible lossy conversion`
Letterale intero troppo grande	`integer number too large`
Riassegnare una costante	`cannot assign a value to final variable`
Stringhe con `==`	compila, ma il confronto è sui riferimenti
Divisione per zero tra interi	`ArithmeticException: / by zero`

DOMANDE?



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.1